Space Complexity

Space complexity measures how much memory an algorithm uses as a function of input size. It is as important as running time for understanding algorithm efficiency, especially in memory-constrained environments.

Measuring Space

Space counted: memory used by the algorithm beyond the input itself.

- **Auxiliary space**: extra variables, data structures, call stack
- **Total space**: input plus auxiliary

Usually, we measure **auxiliary space** to isolate the algorithm's overhead.

Space $S(n)$ is a function of input size n.

Components of Space Usage

Variables and constants: $O(1)$ — bounded regardless of n.

Arrays and lists: $O(k)$ where k is the number of elements stored.

Recursion depth: each function call uses stack space for local variables. Depth d uses $O(d)$ space. - Example: binary search recursion has depth $O(\log n)$, so $S(n) = O(\log n)$ - Example: naive quicksort with bad pivot choice has depth $O(n)$, so $S(n) = O(n)$

Auxiliary data structures: hash tables, trees, queues often store $O(n)$ elements.

Common Space Complexities

- $O(1)$: constant space (in-place operations, a few variables)
  - Example: swap two numbers
- $O(\log n)$: logarithmic (balanced recursion depth)
  - Example: binary search
- $O(n)$: linear (single array or list)
  - Example: merge sort (merges use extra arrays), BFS/DFS (queue/stack stores vertices)
- $O(n^2)$: quadratic (matrix, table of pairs)
  - Example: adjacency matrix for graph

Time-Space Trade-off

Often you can reduce time by using more space, or reduce space by taking more time.

Hash tables: $O(n)$ space for $O(1)$ lookup instead of $O(\log n)$ lookup with just $O(\log n)$ space (balanced tree).

Preprocessing: build an index (use space) to enable fast queries (reduce query time).

Memoization: store computed results (use space) to avoid recomputation (reduce time).

Amortized and Average Space

Some algorithms have variable space usage depending on input:

Hash table: $O(n)$ space for n elements, but with collisions, probing or chaining may temporarily use more space.

Recursion: some algorithms have average recursion depth better than worst case (e.g., quicksort: average $O(\log n)$ depth vs worst $O(n)$).

In-Place Algorithms

An in-place algorithm modifies the input without using extra space proportional to input size. The algorithm uses only $O(1)$ or $O(\log n)$ auxiliary space.

Examples: - [[Insertion Sort]]: rearranges array in place; $O(1)$ auxiliary space - [[Merge Sort]]: requires temporary arrays for merging; $O(n)$ auxiliary space (not in-place) - [[Heaps|Heapsort]]: sorts in place; $O(1)$ auxiliary space

---

Practical Considerations

- **Cache locality**: even if space is "small" asymptotically, poor memory layout (scattered pointers) can be slower than using more contiguous space
- **Memory limits**: in embedded systems or real-time applications, space constraints are critical
- **Recursion depth limits**: very deep recursion (e.g., $O(n)$ depth) can exhaust the call stack

---

Related

- [[Running Time]]
- [[Big-O, Big-Omega, Big-Theta]]
- [[Amortized Analysis]]
- [[Recursion Relations]]